

Gardening System API

Your automated gardening system must expose several public methods that will be invoked by a script to simulate various environmental conditions affecting your garden. Your implementation must ensure the survival of the plants under these conditions. Please ensure that this implementation is standalone, we will not be using the GUI for this portion of the project. Also, please define the methods as shown in this document and make sure that paths for files (i.e Logs, Config Files, etc) are not absolute paths.

```
class GardenSimulationAPI{
```

```
    void initializeGarden()
```

This method initializes the garden with a predefined set of plants. You may define a configuration file with the types and numbers of the plants in your garden based on your defined plant types.

This method is intended to be called first in any simulation run.

Note: While the GUI allows users to add new plants and specify their needs like water requirements and susceptibility to parasites, this method is specifically for seeding data programmatically from a **config file** for testing purposes or further 3rd party automation. Upon initialization, ensure that there are at least 10 plants alive. Ensure that all varieties of plants are alive in the garden at the start.

Ideally this config file should follow this format:

```
{
  "plants": [
    { "name": str, "amount": int },
    { "name": str, "amount": int },
    etc.
  ]
}
```

As an example:

```
{
  "plants": [
    { "name": "Rose", "amount": 5 },
    { "name": "Tomato", "amount": 5 }
  ]
}
```

Map<String, Object> getPlants()

Returns a map containing:

- "plants": a List<String> of plant names,
- "waterRequirement": a List<Integer> detailing how much water each plant needs,
- "parasites": a List<List<String>> listing parasites each plant is vulnerable to.

This method retrieves all plant definitions set in the initializeGarden method. Plants that are dead should be removed from the list.

Example:

```
{
    plants = [Rose, Tomato],
    waterRequirement = [10, 15],
    parasites = [[pest1, pest2], [pest1]]
}
```

void rain(int)

Simulates rainfall in the garden. The automated system should adjust water levels for the plants based on the rainfall amount. This should reset to normal after 1 simulated day if that day isn't also rainy.

Input range: The range will be determined based on the output received from the getPlants.waterRequirement values

void temperature(int)

If your system includes a temperature response module, use this method to receive and handle daily temperature data. This will set the temperature during the day, after the day ends, the temperature should reset.

Input range: 40 to 120 F

void parasite(str)

Triggers a parasite infestation based on each plant's vulnerabilities as defined in the getPlants() method. Pest control responses are allowed, but they should not "heal" the plant back to full health upon deployment. Plants (based on their properties) can heal over time as scientifically and realistically reasonable.

void getState()

Logs details about the garden's current state, including which plants are alive, which have died, and any other relevant data. These logs should be clear. A good example log would be "DAY, EVENT, EVENT_VALUE, PLANTS_ALIVE"

```
}
```

Operational Overview

Upon calling the `initializeGarden()`, the simulation's clock starts, i.e., the beginning of your first day.

Each hour post-initialization represents a new day in the simulation. A script will randomly invoke `rain()`, `temperature()`, or `parasite()` every hour, simulating the passage of days. The health of the plants should not reset after a simulated day. **After 24 simulated hours (i.e., 24 days)**, `getState()` will be invoked to assess the performance of your automated gardening system.

Additionally, don't forget the requirement to maintain a `log.txt` file that records all events.

Here's a pseudo-code of how your application may be tested in a simulation

```
1 public class GardenSimulator {
2     public static void main(String[] args) {
3
4         GardenSimulationAPI gardenAPI = new GardenSimulationAPI();
5
6         // beginning of the simulation
7         gardenAPI.initializeGarden(); // this marks the beginning of the clock
8         HashMap<String, Object> initialPlantDetails = gardenAPI.getPlants();
9
10        gardenAPI.rain(25);
11        sleepOneHour(); // end of first day
12
13        // beginning of the second day
14        gardenAPI.temperature(60);
15        gardenAPI.parasites("insects");
16        sleepOneHour(); // end of second day
17
18        //.... after 24 days
19
20        gardenAPI.getStatus();
21    }
```